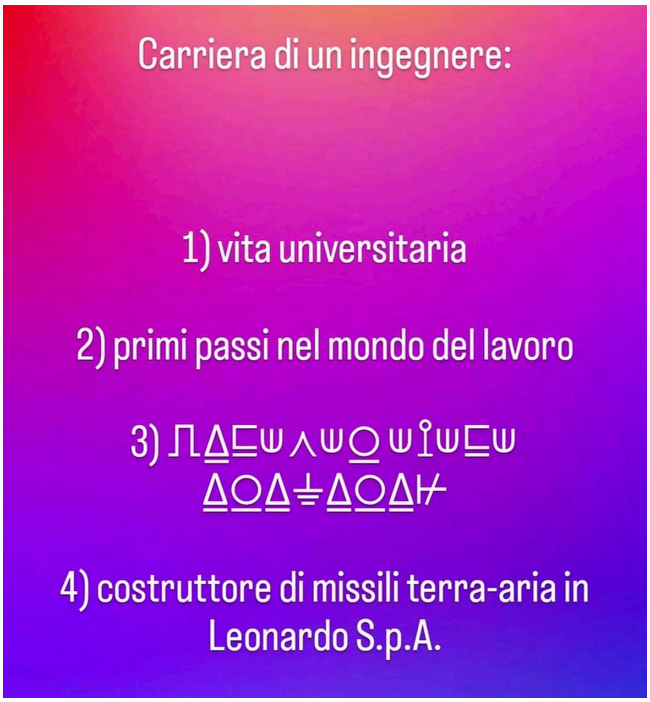
Technische Universität Wien

Computer-Aided Verification

course material: slides



meme translation: “an engineer’s carreer: 1) uni life 2) first steps in the job market 3) [whatever that is] 4) surface-to-air missile builder for Leonardo S.p.A (famous italian defense company with very dubious ethics)”

aglaia norza

October 15, 2026

norzaglaia@gmail.com
github.com/AglaiaNorza

Contents

1 Introduction **2**
1.1 Verification & Validation . . . 2
1.2 Model Checking Basics . . . 2
1.3 The State-Space Explosion Problem . . . 3
1.3.1 Remedies . . . 3
1.4 Symbolic Model Checking . . . 3
1.4.1 Invariants and Fixpoints . . . 4
1.5 Abstraction and CEGAR . . . 4
1.6 Protocols & Execution Models . . . 4

2 Temporal Logics and Model Checking **6**
2.1 Explicit-State Model Checking of Assertions . . . 6
2.2 Temporal Logics . . . 7

CHAPTER 1

Introduction

1.1 | Verification & Validation

Software and integrated circuits are pervasive, and a huge effort is spent on verification and validation (V&V). Approximately 50% of software development time and 35% of hardware development time is dedicated to these tasks. The core goal of Computer-Aided Verification (CAV) is either to formally establish the correctness of a system or to find bugs.

1.2 | Model Checking Basics

Def. 1 **Transition Systems**

Programs and circuits are modeled as **transition systems**.

- **States (S):** represent configurations, such as program counter values (pc) and variable assignments.
- **Transitions (T):** define how states change, essentially capturing the operational semantics of the system by mapping current states to next states ($s \to s'$).

A formal model checking problem can consist of:

- starting states (i): the initial configurations of the system;
- transitions (t): the allowable state updates;
- bad states ($\neg p$): states that violate the specification;

Violations of specifications are witnessed by a **counterexample**.

Safety vs Liveness violations

- **Safety violation:** a counterexample consisting of a path of *finite length* that reaches a bad state.
  - example: ”two intersecting traffic lights must never be green at the same time”
    counterexample: a finite path where the North-South light turns green, a timer glitches, and the East-West light also turns green - the path ends the moment this bad state is reached
- **Liveness violation:** a counterexample consisting of a path of *infinite length* that repeatedly visits bad states and never reaches a good state.
  - example: ”if a pedestrian presses the crosswalk button, the walk signal will eventually turn green”

counterexample: an infinite loop where the pedestrian presses the button, but the traffic lights never turn green

To properly define ”bad” or ”good” states, engineers verify systems against formal specifications written in **Temporal Logic**. This replaces natural language requirements with precise mathematical constraints (e.g., $GFp \rightarrow G(Gq \vee Fr)$).

1.3 | The State-Space Explosion Problem

The main challenge in model checking is the exponential growth of possible states.

Sources of state explosion

- **Data:** enumerating large domains of values, including user inputs;
- **Concurrency:** enumerating all possible process and thread interleaving schedules;
- **Environment:** enumerating all possible states of the execution environment;
- **Faults:** faulty processes that may feed others with junk data;

1.3.1 | Remedies

To mitigate state-space explosion, several techniques are utilized:

- **Abstraction:** mapping concrete values into a coarser, smaller domain;
- **Symbolic techniques:** using Binary Decision Diagrams (BDDs), boolean satisfiability solvers (SAT), and satisfiability-modulo-theory (SMT) solvers to handle sets of states;
- **Partial order reduction:** avoiding the exploration of states via ”redundant paths”;
- **Symmetry reduction:** avoiding the exploration of symmetric states;

1.4 | Symbolic Model Checking

Instead of exploring states one by one (explicit state model checking), symbolic model checking processes sets of states simultaneously.

- **Sets of States** are epresented as *logical formulae* $F(V)$ over program variables, registers, and signals
- **Transitions** are represented as *binary relations* $T(V, V')$ that link current variables V to next-state variables V'

example: reachability

Reachability can be mathematically expressed to check if a property P can be violated in k steps:

$$I(V_0) \wedge \left(\bigwedge_{i=1}^{k} T(V_{i-1}, V_i) \right) \wedge \neg P(V_k)$$

where

- $I(V_0)$ means that V_0 is an initial state
- $\bigwedge_{i=1}^{k} T(V_{i-1}, V_i)$ represents the (valid) transitions
- $\neg P(V_k)$

1.4.1 | Invariants and Fixpoints

When computing reachable states iteratively, we define $R_0 = I$, and subsequent reachable sets as $T(R_{i-1}) = R_i$. The set of all states reachable in up to k steps is $R_{\leq k} = \bigcup_{i=0}^{k} R_i$.

Def. 2 **Safety condition**

A system is considered safe if:

1 | $R_{\leq k}$ contains the initial states I.

2 | T cannot leave $R_{\leq k}$ (the system has reached a **fixpoint**).

3 | $R_{\leq k}$ does not overlap with the bad states $\neg P$.

Since exact reachability computations retard convergence and require complex quantifier alternation ($\forall\exists$), practical approaches often compute over-approximate inductive frames to detect fixpoints more efficiently.

1.5 | Abstraction and CEGAR

Abstraction reduces the state space but introduces non-determinism and may introduce divergent behaviors (non-termination). An abstract program must strictly **over-approximate** the original program, meaning it contains ”more” behaviors than the concrete system.

To handle this automatically, modern model checkers use the **Counterexample-Guided Abstraction Refinement (CEGAR)** loop:

1 | **check abstraction:** see if the abstract model is safe; if safe, the concrete system is safe

2 | **check feasibility:** if a counterexample is found, check if it is feasible in the concrete system; if feasible, a real bug is found

3 | **refine:** if the counterexample is infeasible (a false positive caused by over-approximation), use the failure trace to refine the abstraction and check again

1.6 | Protocols & Execution Models

Verification models differ based on whether they target hardware or software.

- **Concurrent Protocols (software):** standard assumptions include that processes run **asynchronously** (interleave), the scheduler is completely non-deterministic, and individual statements execute atomically; examples include threads accessing shared variables (requiring mutual exclusion algorithms like Peterson’s)
- **Hardware Protocols:** standard assumptions include that components run **synchronously** based on a global clock (every component makes a step at a time) and communicate via actions

or shared memory; examples include cache coherence protocols like MSI (Modified-Shared-Invalid)

CHAPTER 2

Temporal Logics and Model Checking

Def. 3 Finite-State Transition System

A triple (S, T, I) is a **Finite-State Transition System** if

- S is a finite set of states
- $I \subseteq S$ is a set of initial states
- $T \subseteq S \times S$ is a *total* transition relation ($\forall s \in S. \exists s' \in S. T(s, s')$)

Def. 4 Kripke Structure

A quadruple (S, T, I, L) is a **Kripke Structure** if

- (S, T, I, L) is a finite-state transition system
- L is a labelling function $S \to 2^{AP}$(from states to atomic propositions)

Atomic propositions are used to represent properties of states, which can be combined them to generate *assertions*.

We can check whether an assertion holds in a given state via the standard $\vDash$ relation - we write $s \vDash F$ if F holds in state s:

- $s \vDash p \iff p \in L(s)$
- $s \nvDash F \iff p \notin L(s)$ (!! closed world assumption - anything not explicitly defined as true in $L(s)$ is evaluated as false)
- $s \vDash F_1 \vee F_2 \iff s \vDash F_1$ or $s \vDash F_2$
- $s \vDash F_1 \wedge F_2 \iff s \vDash F_1$ and $s \vDash F_2$

Assertions can be used to express that something bad is not supposed to happen (*safety* properties).

For example, if we had a traffic light system, we could write $\texttt{assert}(\neg \texttt{green}_1 \vee \neg \texttt{green}_2)$ to signify that traffic lights 1 and 2 cannot be green at the same time.

2.1 | Explicit-State Model Checking of Assertions

To algorithmically verify if a system design is correct, we can use **explicit-state model checking**. This technique systematically enumerates all reachable states of a finite-state transition system, starting from the initial states I and following the transition relation T, to verify whether a specified property or assertion F holds in every reachable state. The two primary search strategies used to explicitly traverse and evaluate this state space are Depth-First Search (DFS) and Breadth-First Search

(BFS).

Depth-First Search (DFS)
Algorithm ExplicitReachDFS

```
1:  open ← list(I)
2:  visited ← ∅
3:  while open ≠ [] do
4:      s ← head(open)
5:      open ← tail(open)
6:      if s⊭F then
7:          error(s)
8:      end if
9:      for each s′ ∈ S : (s, s′) ∈ T do
10:         if s′ ∉ visited then
11:             visited ← {s′} ∪ visited
12:             open ← s′ :: open          ▷ prepend
13:         end if
14:     end for
15: end while
```

Breadth-First Search (BFS)
Algorithm ExplicitReachBFS

```
1:  open ← list(I)
2:  visited ← ∅
3:  while open ≠ [] do
4:      s ← head(open)
5:      open ← tail(open)
6:      if s⊭F then
7:          error(s)
8:      end if
9:      for each s′ ∈ S : (s, s′) ∈ T do
10:         if s′ ∉ visited then
11:             visited ← {s′} ∪ visited
12:             open ← append(open, s′)        ▷ append
13:         end if
14:     end for
15: end while
```

- keeps only the current stack ($|\text{open}| < |S|$): considered more memory-efficient
- counterexamples can be quite long
- queue grows very large due to frontier size: consumes significantly more memory
- always finds the shortest counterexample

But assertions only allow us to make sure that nothing bad happens: what if we want to guarantee that something *good* happens?

We need to extend our specification language to consider **temporal logics**.

2.2 | Temporal Logics